Project Submission: Stealth Health AI Agent

Next-Generation Personalized Wellness Coaching & Architecture Framework

Shaswat

Type y

1. Project Overview

Stealth Health AI Agent is an AI-powered wellness coaching assistant engineered to deliver hyper-personalized health guidance. The system moves away from rigid, standard chatbots by transitioning seamlessly from unstructured patient onboarding text into an adaptive, multi-turn dialogue. By combining structured information extraction, dynamic protocol-timeline tracking, conversational session memory, and strict protocol-grounded retrieval, the agent delivers highly contextualized behavioral support while safely eliminating the risk of large language model (LLM) hallucinations.

2. Live Application & Artifacts

- **Live Streamlit Application:** stealth-health-ai-agent-coach.streamlit.app
- **GitHub Repository:** github.com/shaswatgithub/Stealth-Health-AI-Agent-coach

URL Query Parameter Onboarding (Demo)

The application natively supports frictionless, link-based onboarding utilizing URL query parameters. Passing text to the `raw_data` parameter automatically triggers automated extraction upon page load, removing initial typing friction for incoming users.

Type your text

- **Example Onboarding URL:**
  `https://stealth-health-ai-agent-coach-ufvt8ajm3ruvedvvtouy9k.streamlit.app/?raw_data=I+am+Guest,+22+years+old.+My+goal+is+improving+sleep.+I+currently+sleep+5+hours.+Put+me+on+Day+5.`

Expected Schema Extraction Result (JSON):

```
{
  "name": "Guest",
  "age": 22,
  "primary_goal": "improving sleep",
  "sleep_hours": 5,
  "current_day": 5
}
```

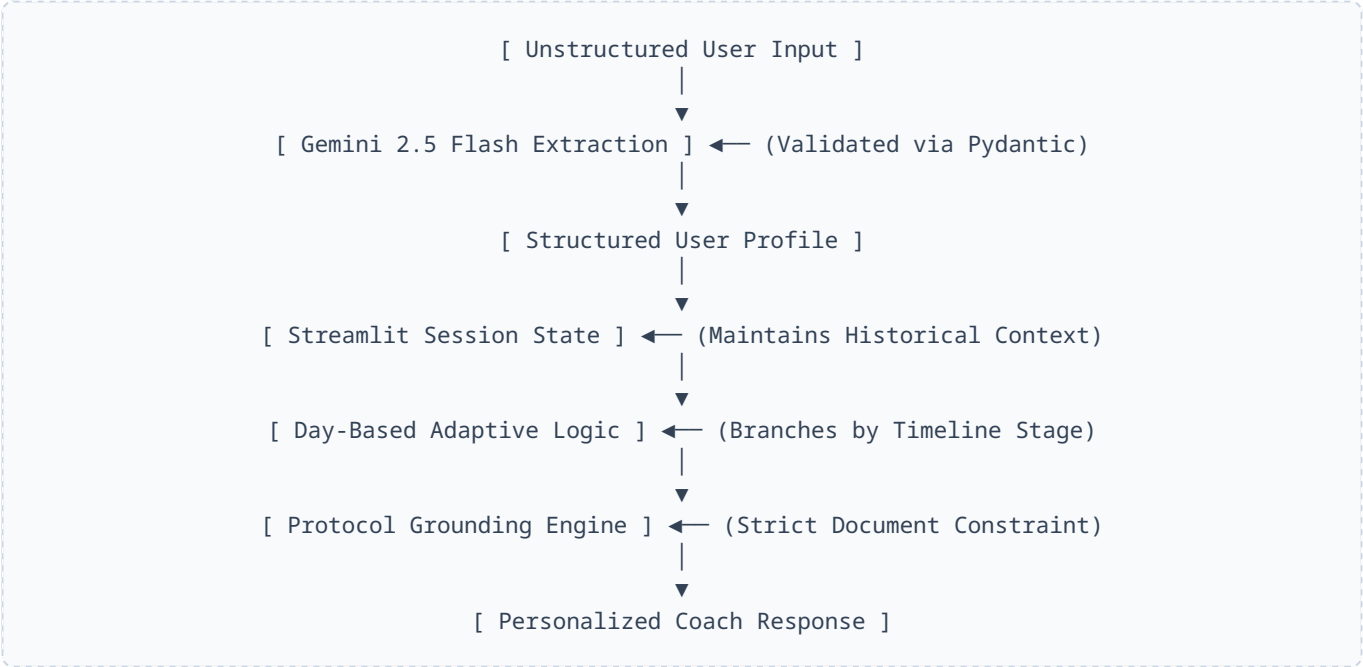
3. Problem Statement

Modern digital wellness platforms struggle to balance automated scale with meaningful personalization. Standard rule-based bots feel clinical and rigid, while open-ended, unconstrained LLMs run the critical risk of hallucinating

health and medical advice. This project develops an intelligent Health Coach Agent that resolves both challenges by hitting four core constraints:

1. **Structured Onboarding:** Seamlessly parses complex, conversational natural language inputs into clean schemas.
2. **Adaptive Progression:** Modifies coaching behaviors and metrics dynamically depending on where the user stands on their protocol timeline.
3. **Strict Grounding:** Confines conversational answers strictly to verified protocol documentation to guarantee domain safety.
4. **Contextual Memory:** Retains conversation state across multiple turns for fluid, high-empathy interactions.

4. Solution Architecture



5. Key System Features

A. Structured Patient Onboarding

The system applies constrained JSON generation to extract critical entities directly from free-form user introductory text, storing them as a structured state: Name, Age, Primary Wellness Goal, Baseline Sleep Hours, and Target Protocol Track Day.

B. Adaptive Daily Check-ins

The agent alters its tone, intent, and inner prompt architecture based on the user's current progress milestone:

- **Day 1 (The Baseline):** Focuses on warm welcomes, validating core motivations, and setting baseline metrics.
- **Days 2–4 (The Consistency Grind):** Focuses on low-friction progress tracking, streak validation, and checking minor consistency metrics.
- **Days 5+ (The Accountability Zone):** Deploys advanced coaching strategies—identifying hidden behavioral friction points, handling complex obstacles, and introducing adaptive behavioral modifications.

C. Protocol-Grounded Guardrails

To eliminate medical misinformation, the reasoning engine passes the core prompt through a defensive grounding layer. If a user asks a question whose answer does not explicitly exist within the reference wellness framework, the agent is restricted from utilizing its base weights and falls back gracefully:

> *"I couldn't find that in the protocol document."*

6. Technology Stack

Component	Selected Technology	Engineering Justification
Frontend UI	Streamlit	Rapid prototyping framework containing built-in low-latency chat interface elements perfect for functional MVPs.
AI Reasoning Engine	Gemini 2.5 Flash	Exceptional native instruction-following capabilities, fast token processing speeds, and cost-efficient structured outputs.
Data Validation	Pydantic	Enforces strict, type-safe data schemas ensuring the model output compiles cleanly before application ingestion.
State Management	Streamlit Session State	Lightweight, transactional in-memory context retention without requiring heavy database read/write configurations.
Hosting & CI/CD	Streamlit Community Cloud	Native git-integrated continuous deployment pipeline for seamless repository-to-production tracking.

7. Technical Decisions & Trade-offs

- **Pydantic Integration over Raw Text Parsing:** Relying on regular expressions or simple string splitting to parse onboarding text breaks under conversational variances. Forcing Gemini to conform to a Pydantic schema guarantees that our runtime application never encounters unexpected types or missing keys.
- **In-Memory Session State vs. External Database:** For this phase of the MVP, utilizing local application session state removes network latency and database hosting complexities. It satisfies multi-turn conversational goals perfectly for single-session verification, though it clears upon browser refresh.

8. Suggested Evaluation Flow

To evaluate the live application comprehensively, execute the following script during the assessment:

1. **Direct Navigation:** Load the web application homepage to observe the raw onboarding container interface.
2. **Query Parameter Injection:** Paste the demo URL into the browser to confirm instant parsing of names, goals, and targets without requiring user typing.
3. **Day 1 Verification:** Initiate a conversation configured to *Day 1* to evaluate the intake and baseline setup dialogue.

4. **Day 5 Accountability Shift:** Change the state to *Day 5* to verify the shift into deep obstacle identification and behavioral adjustments.
5. **Grounding Check:** Ask a completely out-of-bounds question (e.g., *"How do I fix a car radiator?"*) to verify that the guardrails intercept the text with the designated protocol error message.

9. Future Development Roadmap

- **Vector-Based Retrieval Augmented Generation (RAG):** Transitioning the static grounding module into an active vector storage index (e.g., ChromaDB or Pinecone) to support complex, multi-page PDF protocol guidebooks.
- **Persistent User Profiles:** Layering PostgreSQL or Firebase authentication underneath to store user profiles and track metrics seamlessly over weeks instead of single sessions.
- **Analytics Visualizations:** Injecting interactive data charting to map reported sleep hours against protocol days to visually show historical progress trends.